

Taller 1: Complejidad

Isabella Hermosa Losada¹

¹ Pontificia Universidad Javeriana, Bogotá D.C., Colombia
ishermosa@javeriana.edu.co

1 Análisis inicial

1.1 Organización del código

La programación modular consiste en dividir un programa en partes más pequeñas llamadas módulos, donde cada una se encarga de una tarea específica. Esta organización permite que el código sea más legible y fácil de recorrer, ya que, cuando un proyecto es extenso, tener toda la información en un único archivo fuente dificulta encontrar rápidamente una función, entender el funcionamiento del programa o detectar en qué parte se encuentra un error. Además, permite agregar nuevas funcionalidades sin tener que reescribir todo el código, lo que facilita la escalabilidad del proyecto [1].

La división en varios archivos también favorece la reutilización y reduce la redundancia, entendida como la repetición innecesaria de declaraciones, definiciones o partes de la lógica del código [3]. Al separar las declaraciones de las implementaciones, una misma declaración puede ser utilizada por diferentes archivos sin tener que escribirla nuevamente en cada uno. Esto evita inconsistencias y permite que las funciones puedan reutilizarse en otras partes del programa o incluso en otros proyectos [2].

Finalmente, esta organización facilita la compilación y el mantenimiento. Cada archivo fuente puede compilarse como una unidad separada y luego unirse con los demás durante el enlazado para generar el ejecutable final [2]. Por esta razón, cuando se necesita corregir una función, mejorar su rendimiento o modificar una parte del programa, es más fácil identificar el archivo correspondiente y realizar el cambio sin afectar innecesariamente todo el proyecto.

1.2 Tipos de archivos de código

Los archivos con extensión `.h`, también conocidos como archivos de encabezado o *headers*, contienen principalmente las declaraciones que otros archivos necesitan conocer, como funciones, clases, estructuras, variables y constantes. Su función es indicar qué elementos están disponibles sin incluir necesariamente la lógica completa de su funcionamiento. Estas declaraciones pueden compartirse con distintos archivos fuente mediante la instrucción `#include` [2].

Por otra parte, los archivos `.cxx` son archivos fuente escritos en C++ y normalmente contienen las implementaciones de las funciones declaradas en los encabezados, además de la lógica del programa. La extensión `.cxx` cumple la misma función que otras extensiones como `.cpp` o `.cc`, por lo que su elección depende de la convención adoptada en cada proyecto [4].

En este taller, por ejemplo, `suma_functions.h` contiene las declaraciones de las funciones de suma, mientras que `suma_functions.cxx` contiene sus implementaciones.

Los archivos `.hxx` son otro tipo de archivo de encabezado utilizado en C++. No existe una diferencia técnica obligatoria entre las extensiones `.h` y `.hxx`, pero algunos proyectos utilizan `.hxx` para separar elementos propios de C++, como la implementación de plantillas o *templates*.

Las plantillas tienen una condición especial: el compilador necesita conocer su implementación completa en el momento en que se utilizan para poder generar la función correspondiente. Por esta razón, su implementación normalmente debe estar disponible para el compilador y no quedar únicamente en un archivo `.cxx` compilado por separado [5].

En este proyecto, `sort_functions.h` contiene las declaraciones de `bubblesort`, `quicksort`, `heapsort` y de la plantilla `is_sorted`. Por otro lado, `sort_functions.hxx` guarda la implementación de `is_sorted`. Al final de `sort_functions.h` se incluye el archivo `.hxx` mediante:

```
#include "sort_functions.hxx"
```

Esto permite que el compilador pueda leer la implementación completa de la plantilla cuando la necesite.

Durante la compilación, cada archivo `.cxx`, junto con los archivos `.h` y `.hxx` que incluye, forma una unidad de traducción que se procesa por separado. Después, el enlazador reúne los archivos compilados, relaciona las llamadas a las funciones con sus implementaciones y genera el ejecutable final [6].

Por esta razón, en los comandos del taller se escriben los archivos `.cxx` que deben compilarse juntos, mientras que los archivos `.h` y `.hxx` no se agregan directamente al comando, ya que su contenido se incorpora mediante las instrucciones `#include`.

1.3 Archivos de prueba

Los archivos que comienzan con `test_` son `test_suma.cxx`, `test_sort_random.cxx`, `test_sort_sorted.cxx` y `test_sort_inverse.cxx`. Cada uno contiene su propia función `main`, por lo que funciona como un programa independiente encargado de recibir datos, ejecutar los algoritmos, comprobar sus resultados y medir sus tiempos de ejecución.

Todos estos programas reciben sus datos desde la terminal mediante `argc` y `argv`. `argc` indica cuántos elementos fueron escritos en el comando y `argv` los almacena como cadenas de texto. En esta estructura, `argv[0]` contiene el nombre del ejecutable, mientras que las siguientes posiciones contienen los valores ingresados por el usuario. La función `std::atoi` convierte estos valores de texto a números para que puedan utilizarse dentro del programa.

En `test_suma.cxx`, la condición `argc < 3` comprueba que se hayan ingresado los dos argumentos requeridos. Si falta alguno, el programa muestra la forma correcta de usar el ejecutable y termina mediante `return -1`. Después, `argv[1]` se convierte en

`n`, que representa el límite superior de la suma, y `argv[2]` se convierte en `k`, que indica el número cuyos múltiplos se deben sumar.

También se verifica que ambos valores sean mayores que cero. Si alguno es igual a cero, se muestra un mensaje de error y el programa finaliza sin ejecutar los algoritmos.

Una vez validadas las entradas, `test_suma.cxx` ejecuta `suma_recursive`, `suma_iterative` y `suma_constant` utilizando los mismos valores de `n` y `k`. Las tres funciones deben obtener el mismo resultado cuando están correctamente implementadas, pero utilizan métodos diferentes, lo que permite comparar sus tiempos de ejecución.

Para cada función se guarda el instante inicial, se ejecuta el algoritmo, se guarda el instante final y se calcula la diferencia. En este programa los tiempos se registran en nanosegundos. Finalmente, se imprimen ocho campos separados por punto y coma: `n`, `k`, los tres resultados obtenidos y los tres tiempos de ejecución.

Los archivos `test_sort_random.cxx`, `test_sort_sorted.cxx` y `test_sort_inverse.cxx` reciben un único argumento llamado `count`, que representa el tamaño del arreglo. En estos programas, la condición `argc < 2` comprueba que el usuario haya escrito ese valor. Si no se proporciona, se muestra un mensaje de uso y el programa termina con `return -1`. Luego, `argv[1]` se convierte a un valor de tipo `long`.

Después se reserva memoria dinámica para tres arreglos llamados `a_bubble`, `a_quick` y `a_heap`. Se crean tres arreglos porque cada algoritmo modifica los datos al ordenarlos y, para que la comparación sea justa, BubbleSort, QuickSort y HeapSort deben comenzar con exactamente los mismos valores.

La diferencia entre los tres archivos se encuentra en la forma en que se llenan los arreglos. `test_sort_random.cxx` genera un valor aleatorio en cada posición y lo copia en los tres arreglos. `test_sort_sorted.cxx` crea tres arreglos iguales y los llena con valores ordenados crecientemente desde 0 hasta `count - 1`. Por su parte, `test_sort_inverse.cxx` utiliza la expresión `count - c`, creando una secuencia de mayor a menor.

Luego, cada programa ejecuta BubbleSort, QuickSort y HeapSort por separado. Antes y después de cada algoritmo se registra el tiempo mediante `std::chrono`, y la diferencia obtenida corresponde al tiempo de ejecución en milisegundos.

Al terminar cada ordenamiento, se utiliza `is_sorted` para revisar el arreglo completo y comprobar si quedó organizado correctamente. Esta función produce 1 cuando el arreglo está ordenado y 0 cuando no lo está.

La salida de estos tres programas contiene siete campos separados por punto y coma: el tamaño del arreglo, las verificaciones de BubbleSort, QuickSort y HeapSort, y los tiempos de los tres algoritmos. Después de imprimir los resultados, se libera la memoria reservada mediante `delete[]` y el programa termina correctamente con `return 0`.

Estos archivos permiten comprobar si los algoritmos producen resultados correctos y comparar su rendimiento bajo diferentes tamaños y formas de entrada. Gracias a ellos se obtienen los datos necesarios para realizar el análisis empírico de complejidad temporal y posteriormente relacionarlo con la complejidad teórica.

1.4 Tipos de comandos y sintaxis

En el archivo `comandos.txt` se encuentran tres tipos generales de comandos: compilación, ejecución automática y redirección de salida. Como el taller fue desarrollado

en Ubuntu mediante WSL, se analizaron únicamente los comandos correspondientes a Linux.

Comandos de compilación Los comandos de compilación utilizan `g++` para procesar los archivos fuente y generar un ejecutable. Por ejemplo:

```
g++ -std=c++17 suma_functions.cxx test_suma.cxx -o my_test_suma
```

En este comando, `g++` corresponde al compilador de C++, mientras que `-std=c++17` indica que el programa debe compilarse utilizando el estándar C++17. Los archivos `suma_functions.cxx` y `test_suma.cxx` son los archivos fuente que deben procesarse conjuntamente.

La opción `-o` permite asignar un nombre al ejecutable generado. En este caso, el resultado de la compilación recibe el nombre `my_test_suma` [4]. Los comandos utilizados para los algoritmos de ordenamiento siguen la misma estructura, pero cambian los archivos fuente y el nombre del ejecutable correspondiente.

Comandos de ejecución automática El segundo tipo corresponde a comandos que permiten ejecutar un programa varias veces sin tener que escribir cada valor manualmente. Por ejemplo:

```
for i in $(seq 1 1000 65535); do ./my_test_suma $i 2; done
```

Este comando utiliza un ciclo `for` de Bash para repetir la ejecución del programa [7]. La instrucción

```
seq 1 1000 65535
```

genera una secuencia que comienza en 1, aumenta de 1000 en 1000 y termina en el último valor que no supere 65535 [8].

En cada repetición, el valor actual de la secuencia se guarda en la variable `i`. Luego, `$i` se utiliza como el valor de `n`, mientras que el valor 2 permanece fijo como `k`.

La expresión

```
./my_test_suma
```

indica que el ejecutable se encuentra en la carpeta actual. Los otros comandos de suma repiten el mismo procedimiento utilizando valores fijos de `k = 3` y `k = 10`, lo que permite observar cómo cambian los resultados y los tiempos conforme aumenta `n`.

En los comandos de ordenamiento también se utiliza un ciclo para variar el tamaño del arreglo. Por ejemplo:

```
seq 1000 400 20000
```

genera valores desde 1000 hasta 20000 con incrementos de 400. Después, el programa correspondiente se ejecuta utilizando el valor actual de `i` como tamaño del arreglo.

En algunos comandos aparece la sintaxis `$(...)` y en otros se utilizan comillas invertidas. Ambas permiten utilizar el resultado producido por un comando dentro de otro comando de Bash [7].

Redirección de salida El tercer tipo corresponde a la redirección de la salida utilizando el símbolo `>`. Por ejemplo:

```
> suma_table_k2.csv
```

Este símbolo hace que la información que normalmente aparecería en la terminal sea almacenada directamente en un archivo. Si el archivo no existe, se crea; si ya existe, su contenido anterior es reemplazado por los nuevos resultados [7].

Los archivos `.csv` contienen los resultados obtenidos para las funciones de suma, mientras que los archivos `.txt` contienen los resultados de los algoritmos de ordenamiento. Cada línea corresponde a una ejecución diferente y almacena los valores separados mediante punto y coma.

1.5 Archivos generados

Antes de ejecutar los comandos, la carpeta del proyecto contenía únicamente los archivos fuente, los encabezados, el archivo `comandos.txt` y el enunciado del taller, como se observa en la Figura 1. Después de realizar la compilación y las ejecuciones automáticas aparecieron diez archivos nuevos, divididos en ejecutables y archivos de resultados.

Los ejecutables generados fueron `my_test_suma`, `my_test_sort_random`, `my_test_sort_sorted` y `my_test_sort_inverse`. Estos archivos no tienen extensión porque fueron creados y ejecutados dentro del entorno Linux de Ubuntu mediante WSL. Sus nombres permiten identificar la prueba que realiza cada uno.

También se generaron los archivos `suma_table_k2.csv`, `suma_table_k3.csv` y `suma_table_k10.csv`. El nombre `suma_table` indica que corresponden a tablas de resultados de las funciones de suma, mientras que `k2`, `k3` y `k10` permiten reconocer el valor fijo de k utilizado en cada ejecución. La extensión `.csv` permite almacenar los resultados separados mediante delimitadores, facilitando su importación posterior en programas como Excel.

Finalmente, se crearon `sort_random_table.txt`, `sort_sorted_table.txt` y `sort_inverse_table.txt`. La palabra `sort` señala que se relacionan con los algoritmos de ordenamiento, mientras que `random`, `sorted` e `inverse` identifican la organización inicial de los arreglos. La extensión `.txt` indica que sus resultados fueron almacenados como texto plano.

Después de realizar la compilación y ejecutar los comandos automáticos, la carpeta contiene tanto los ejecutables como los archivos utilizados para guardar los resultados obtenidos, como se observa en la Figura 2.

1.6 Contenido y propósito de los archivos

Los archivos generados guardan los datos numéricos obtenidos durante las ejecuciones automáticas de los programas. Cada fila corresponde a un caso de prueba diferente, ya que en cada repetición del ciclo `for` cambia el tamaño de la entrada.

En `suma_table_k2.csv`, `suma_table_k3.csv` y `suma_table_k10.csv` se encuentran los valores de n y k , los resultados de las versiones recursiva, iterativa y constante, y sus respectivos tiempos de ejecución en nanosegundos.

```

C:\Users\Usuario>wsl
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario$ cd Desktop/
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop$ cd talleresicorte/
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte$ cd taller_1_complejidad_archivos/
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte/taller_1_complejidad_archivos$ code .
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte/taller_1_complejidad_archivos$ ls -lh
total 296K
-rwxrwxrwx 1 isa21087 isa21087 3.2K Aug 1 22:40 comandos.txt
-rwxrwxrwx 1 isa21087 isa21087 3.1K Aug 1 22:40 sort_functions.cxx
-rwxrwxrwx 1 isa21087 isa21087 1.8K Aug 1 22:40 sort_functions.h
-rwxrwxrwx 1 isa21087 isa21087 408 Aug 4 12:01 sort_functions.hxx
-rwxrwxrwx 1 isa21087 isa21087 1.9K Aug 1 22:40 suma_functions.cxx
-rwxrwxrwx 1 isa21087 isa21087 1.7K Aug 1 22:40 suma_functions.h
-rwxrwxrwx 1 isa21087 isa21087 250K Aug 1 22:05 taller_1_complejidad.pdf
-rwxrwxrwx 1 isa21087 isa21087 3.4K Aug 4 12:01 test_sort_inverse.cxx
-rwxrwxrwx 1 isa21087 isa21087 3.4K Aug 1 22:40 test_sort_random.cxx
-rwxrwxrwx 1 isa21087 isa21087 3.3K Aug 1 22:40 test_sort_sorted.cxx
-rwxrwxrwx 1 isa21087 isa21087 5.3K Aug 4 12:01 test_suma.cxx
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte/taller_1_complejidad_archivos$

```

Fig. 1. Archivos originales presentes en la carpeta antes de compilar y ejecutar los comandos del taller. Fuente: captura propia.

```

isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte/taller_1_complejidad_archivos$ ls -lh
-rwxrwxrwx 1 isa21087 isa21087 23K Aug 5 22:50 my_test_sort_inverse
-rwxrwxrwx 1 isa21087 isa21087 23K Aug 5 22:49 my_test_sort_random
-rwxrwxrwx 1 isa21087 isa21087 23K Aug 5 22:49 my_test_sort_sorted
-rwxrwxrwx 1 isa21087 isa21087 17K Aug 5 22:34 my_test_suma
-rwxrwxrwx 1 isa21087 isa21087 1.1K Aug 5 23:00 sort_inverse_table.txt
-rwxrwxrwx 1 isa21087 isa21087 1007 Aug 5 22:56 sort_random_table.txt
-rwxrwxrwx 1 isa21087 isa21087 1.1K Aug 5 22:59 sort_sorted_table.txt
-rwxrwxrwx 1 isa21087 isa21087 3.4K Aug 5 22:54 suma_table_k10.csv
-rwxrwxrwx 1 isa21087 isa21087 3.4K Aug 5 22:51 suma_table_k2.csv
-rwxrwxrwx 1 isa21087 isa21087 3.4K Aug 5 22:51 suma_table_k3.csv
isa21087@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/talleresicorte/taller_1_complejidad_archivos$

```

Fig. 2. Archivos ejecutables y archivos de resultados generados después de ejecutar los comandos en Ubuntu mediante WSL. Fuente: captura propia.

Al revisar los archivos generados inicialmente, se observa que las versiones recursiva e iterativa coinciden, mientras que la versión constante devuelve siempre el valor 1. Esto ocurre porque, en este punto inicial del taller, la fórmula correspondiente a `suma_constant` todavía no había sido implementada. Por esta razón, estos archivos debían generarse nuevamente después de completar dicha función.

En la Figura 3 se observan las primeras filas de uno de estos archivos.

Por otro lado, `sort_random_table.txt`, `sort_sorted_table.txt` y `sort_inverse_table.txt` contienen el tamaño del arreglo, la verificación de que cada algoritmo lo haya ordenado correctamente y los tiempos de ejecución de BubbleSort, QuickSort y HeapSort en milisegundos.

La diferencia entre estos archivos se encuentra en el orden inicial de los datos. En `sort_random_table.txt` los valores se generan aleatoriamente, en `sort_sorted_table.txt` ya se encuentran ordenados de menor a mayor y en `sort_inverse_table.txt` aparecen organizados de forma inversa.

Un ejemplo del contenido generado para las pruebas de ordenamiento se presenta en la Figura 4.

La diferencia encontrada en los resultados de las funciones de suma se puede relacionar directamente con el estado inicial de `suma_constant`. Como se muestra en la Figura 5, la función retornaba directamente el valor 1 antes de incorporar la fórmula matemática requerida.

```

suma_table.k2.csv > data
1  ex[633];for i in $(seq 1 1000 65535);09017bf2-9c14-412a-ab67-55ba51c2abec;ex[633];Cm[1;2;0;0;1;475;113;61;
2  1001;2;2.50500e+07;2.50500e+07;1.20702;2633;67;
3  2001;2;1.001e+06;1.001e+06;1.17460;5345;71;
4  3001;2;2.2515e+06;2.2515e+06;1.10118;8529;80;
5  4001;2;4.002e+06;4.002e+06;1.14224;11742;83;
6  5001;2;6.2525e+06;6.2525e+06;1.176206;13613;72;
7  6001;2;9.003e+06;9.003e+06;1.197480;15303;71;
8  7001;2;1.22535e+07;1.22535e+07;1.230017;17939;72;
9  8001;2;1.6004e+07;1.6004e+07;1.343202;21638;91;
10 9001;2;2.02545e+07;2.02545e+07;1.397559;25635;95;
11 10001;2;2.5005e+07;2.5005e+07;1.479084;26630;96;
12 11001;2;3.02555e+07;3.02555e+07;1.494486;29475;172;
13 12001;2;3.6006e+07;3.6006e+07;1.394412;34646;71;
14 13001;2;4.22565e+07;4.22565e+07;1.430555;32774;71;
15 14001;2;4.9007e+07;4.9007e+07;1.526724;33938;74;
16 15001;2;5.62575e+07;5.62575e+07;1.473303;36380;69;
17 16001;2;6.4008e+07;6.4008e+07;1.521457;38743;71;
18 17001;2;7.22585e+07;7.22585e+07;1.585877;44701;73;
19 18001;2;8.1009e+07;8.1009e+07;1.783333;47830;199;
20 19001;2;9.02595e+07;9.02595e+07;1.618876;45970;61;
21 20001;2;1.0001e+08;1.0001e+08;1.966142;50336;68;
22 21001;2;1.1026e+08;1.1026e+08;1.747199;71082;104;
23 22001;2;1.21011e+08;1.21011e+08;1.799109;53161;71;
24 23001;2;1.32262e+08;1.32262e+08;1.833136;55577;67;
25 24001;2;1.44012e+08;1.44012e+08;1.813251;58005;66;
26 25001;2;1.56262e+08;1.56262e+08;1.878815;62880;78;
27 26001;2;1.69013e+08;1.69013e+08;1.2780318;69814;161;
28 27001;2;1.82264e+08;1.82264e+08;1.993970;68011;119;
29 28001;2;1.96014e+08;1.96014e+08;1.1246870;70462;70;
30 29001;2;2.10264e+08;2.10264e+08;1.1123590;81254;107;
31 30001;2;2.25015e+08;2.25015e+08;1.1174080;77078;71;
32 31001;2;2.40266e+08;2.40266e+08;1.1277015;85018;82;

```

Fig. 3. Primeras filas del archivo suma_table.k2.csv, donde se observan los valores de entrada, los resultados de las tres versiones y sus tiempos de ejecución. Fuente: elaboración propia.

```

sort_random_table.txt
1  ex[633];for i in `seq 1000 400 20000`;09017bf2-9c14-412a-ab67-55ba51c2abec;ex[633];Cm[1000;1;1;1;4;0;0;
2  1400;1;1;1;6;0;0;
3  1800;1;1;1;8;1;0;
4  2200;1;1;1;12;0;0;
5  2600;1;1;1;21;0;0;
6  3000;1;1;1;23;0;1;
7  3400;1;1;1;34;1;1;
8  3800;1;1;1;34;1;0;
9  4200;1;1;1;43;0;1;
10 4600;1;1;1;50;0;1;
11 5000;1;1;1;64;1;1;
12 5400;1;1;1;73;0;1;
13 5800;1;1;1;80;1;1;
14 6200;1;1;1;96;0;2;
15 6600;1;1;1;108;1;1;
16 7000;1;1;1;128;2;2;
17 7400;1;1;1;180;0;2;
18 7800;1;1;1;160;1;2;
19 8200;1;1;1;174;1;2;
20 8600;1;1;1;188;1;1;
21 9000;1;1;1;218;1;2;
22 9400;1;1;1;241;1;2;
23 9800;1;1;1;266;1;2;
24 10200;1;1;1;300;1;2;
25 10600;1;1;1;327;1;3;
26 11000;1;1;1;344;2;3;
27 11400;1;1;1;484;2;3;
28 11800;1;1;1;382;2;2;
29 12200;1;1;1;470;1;3;
30 12600;1;1;1;487;2;2;
31 13000;1;1;1;470;2;3;
32 13400;1;1;1;677;1;3;

```

Fig. 4. Primeras filas del archivo sort_random_table.txt, donde se observan el tamaño del arreglo, las verificaciones de ordenamiento y los tiempos de ejecución. Fuente: elaboración propia.

```

double suma_constant(unsigned short n, unsigned short k) {
    // TODO #01: Implementar esta función utilizando una fórmula matemática
    // para calcular directamente la suma de los múltiplos de k menores o iguales a n.
    return 1; // Valor de retorno provisional
}

// eof - suma_functions.cxx

```

Fig. 5. Implementación inicial de `suma_constant`, la cual retornaba el valor 1 antes de incorporar la fórmula matemática. Fuente: elaboración propia.

La finalidad de estos archivos es reunir los resultados de las pruebas para analizar de forma empírica cómo cambia el tiempo de ejecución de cada algoritmo cuando aumenta el tamaño de la entrada. Además, permiten comprobar si las distintas implementaciones producen los resultados esperados antes de realizar el análisis de complejidad.

2 Cálculo de la suma de los múltiplos de un número k menores o iguales a un número n

2.1 Demostración e implementación de la versión constante

Para obtener una forma directa de calcular la suma de los múltiplos de k menores o iguales a n , primero se debe determinar cuántos múltiplos de k existen dentro de este intervalo.

Si el último múltiplo de k que no supera a n corresponde a mk , entonces la cantidad de múltiplos puede expresarse como

$$m = \left\lfloor \frac{n}{k} \right\rfloor. \quad (1)$$

Los múltiplos que deben sumarse pueden escribirse como

$$k, 2k, 3k, \dots, mk.$$

Por lo tanto, la suma puede expresarse inicialmente como

$$S = k + 2k + 3k + \dots + mk.$$

Al sacar k como factor común se obtiene

$$S = k(1 + 2 + 3 + \dots + m). \quad (2)$$

La expresión que aparece dentro del paréntesis corresponde a la suma de los primeros m números naturales. Este tipo de suma puede obtenerse a partir de las propiedades de las sucesiones y series aritméticas [9,10,11]:

$$1 + 2 + 3 + \dots + m = \frac{m(m+1)}{2}.$$

A partir de lo anterior, la suma de los múltiplos de k se puede expresar como

$$S = k \left(\frac{m(m+1)}{2} \right). \quad (3)$$

Finalmente, se reemplaza m por el valor definido en la ecuación 1, obteniendo

$$S = k \left(\frac{\lfloor n/k \rfloor (\lfloor n/k \rfloor + 1)}{2} \right). \quad (4)$$

De esta manera, la expresión permite calcular directamente la suma de todos los múltiplos de k menores o iguales a n .

Ejemplo de aplicación Para comprobar la fórmula, se puede tomar como ejemplo $n = 10$ y $k = 3$. Primero se determina la cantidad de múltiplos:

$$m = \left\lfloor \frac{10}{3} \right\rfloor = 3.$$

Esto significa que existen tres múltiplos de 3 menores o iguales a 10: 3, 6 y 9. Luego, al aplicar la ecuación 3,

$$S = 3 \left(\frac{3(3+1)}{2} \right),$$

se obtiene

$$S = 18.$$

El resultado coincide con la suma directa de los múltiplos:

$$3 + 6 + 9 = 18.$$

Por lo tanto, para este caso la fórmula cerrada produce el mismo resultado que realizar la suma término por término.

Implementación de la fórmula Una vez obtenida la expresión matemática, se procedió a implementarla en la función `suma_constant`.

En el código, la variable `m` almacena la cantidad de múltiplos de k que existen hasta n . Debido a que n y k son valores enteros, la operación n / k conserva únicamente la parte entera del resultado, que en este caso corresponde a

$$\left\lfloor \frac{n}{k} \right\rfloor.$$

Posteriormente, este valor se utiliza para aplicar directamente la expresión de la ecuación 3. A diferencia de las versiones recursiva e iterativa, esta implementación no necesita recorrer uno por uno los valores hasta n , ya que el resultado se obtiene directamente mediante operaciones aritméticas.

La implementación realizada en el archivo `suma_functions.cxx` se muestra en la Figura 6.

```
//
double suma_constant(unsigned short n, unsigned short k) {
    // TODO #01: Implementar esta función utilizando una fórmula matemática
    // para calcular directamente la suma de los múltiplos de k menores o iguales a n.
    double m = n / k;
    return k * m * (m + 1) / 2;
}
```

Fig. 6. Implementación de la fórmula cerrada en la función `suma_constant` del archivo `suma_functions.cxx`. Fuente: elaboración propia.

2.2 Análisis gráfico

En la gráfica correspondiente a $k = 2$ se observa que la versión recursiva presenta los tiempos de ejecución más altos y una tendencia general creciente a medida que aumenta n . Además, aparecen algunos picos que se alejan de la tendencia principal. Debido a la diferencia tan grande entre esta versión y las otras dos, la escala de la gráfica hace que la versión iterativa y, especialmente, la constante parezcan casi pegadas al eje horizontal.

Sin embargo, al observar únicamente los datos de la versión constante se evidencia que su tiempo no es exactamente igual en todas las ejecuciones. La mayoría de los valores se mantienen muy bajos, generalmente alrededor de decenas de nanosegundos, aunque aparecen algunos valores mayores. Incluso el valor más alto observado, cercano a los 500 ns, sigue siendo demasiado pequeño en comparación con los tiempos de las versiones iterativa y recursiva.

Por esta razón, no se puede afirmar que el tiempo medido sea completamente igual en todas las pruebas, pero sí que se mantiene bastante estable respecto al crecimiento de n .



Fig. 7. Comparación de los tiempos de ejecución para $k = 2$. Fuente: elaboración propia.

Para $k = 3$, la versión recursiva continúa presentando los mayores tiempos de ejecución. A medida que aumenta n , se observa una tendencia general creciente, pero acompañada de fluctuaciones cada vez más marcadas, especialmente en los valores más altos de entrada.

Por esta razón, no se observa que cada aumento de n produzca necesariamente un tiempo mayor al anterior, ya que existen puntos donde el tiempo disminuye nuevamente.

La versión iterativa mantiene tiempos considerablemente menores que la recursiva y presenta un crecimiento mucho más moderado. La versión constante permanece en valores muy bajos durante casi toda la ejecución, aunque aparece un valor atípico que supera momentáneamente el tiempo de la versión iterativa.

Al revisar los datos de forma individual, este punto resulta ser una excepción frente al comportamiento general, ya que el resto de los tiempos de la versión constante se mantienen cercanos entre sí.

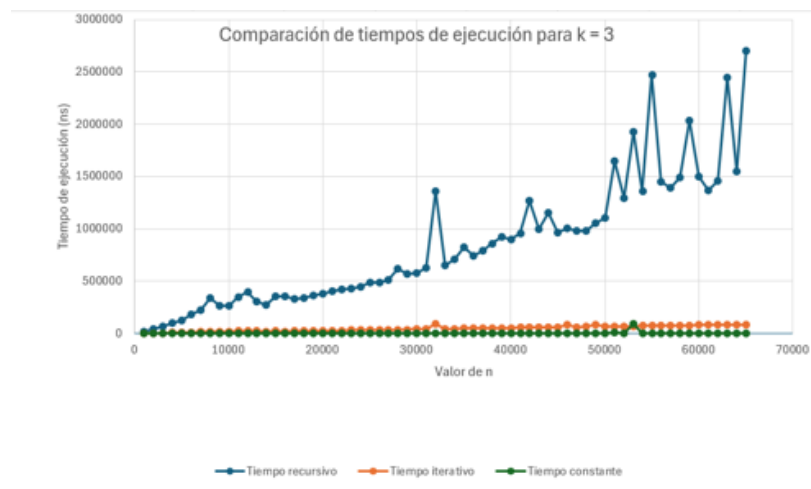


Fig. 8. Comparación de los tiempos de ejecución para $k = 3$. Fuente: elaboración propia.

Finalmente, para $k = 10$ se mantiene la misma diferencia general entre las tres versiones. La recursiva sigue siendo la de mayor tiempo de ejecución y presenta una tendencia creciente con varios picos y descensos. La versión iterativa continúa muy por debajo de esta y muestra también algunas variaciones a medida que aumenta n .

En el caso de la versión constante, al observar la gráfica completa parece mantenerse prácticamente sobre el eje horizontal debido a la escala generada por los tiempos de la versión recursiva. Sin embargo, al analizar sus datos por separado también aparecen pequeñas variaciones y algunos picos.

Aun así, estos cambios son muy pequeños frente a los tiempos de las otras dos versiones y no muestran una relación clara en la que un aumento de n produzca necesariamente un aumento proporcional del tiempo de ejecución.



Fig. 9. Comparación de los tiempos de ejecución para $k = 10$. Fuente: elaboración propia.

En conjunto, las tres gráficas muestran que la versión recursiva presenta el crecimiento más acelerado conforme aumenta n , mientras que la versión constante mantiene los tiempos más bajos y relativamente estables. La versión iterativa se encuentra entre ambas, con un crecimiento visible pero mucho menor que el de la recursiva.

Al comparar los diferentes valores de k , se conserva la misma tendencia general, aunque cambian los tiempos registrados y aparecen diferentes picos en cada ejecución. Debido a la gran diferencia de escala con la versión recursiva, las variaciones de la versión constante casi no son visibles en las gráficas completas.

Sin embargo, al observar sus datos por separado se identifican pequeñas fluctuaciones y algunos valores atípicos, sin evidenciar un crecimiento sostenido conforme aumenta n .

2.3 Comparación de resultados y correctitud

Para comprobar el funcionamiento de las tres versiones del algoritmo, se realizaron pruebas con valores cercanos al límite permitido por el tipo de dato utilizado inicialmente en el programa. Primero se utilizó $n = 65535$ con los valores $k = 2$, $k = 3$ y $k = 10$.

Para $n = 65535$ y $k = 2$, las versiones recursiva, iterativa y constante obtuvieron el mismo resultado, 1.07371×10^9 . Para $k = 3$ también coincidieron, con un resultado de 7.15839×10^8 y, finalmente, para $k = 10$ las tres obtuvieron 2.14742×10^8 .

Por lo tanto, para estas entradas las tres implementaciones continúan produciendo el mismo resultado, aunque sus tiempos de ejecución sean diferentes.

Posteriormente, se realizó la misma prueba utilizando $n = 65536$. En este caso, para los tres valores de k utilizados, el programa mostró el mensaje:

Error: n y k deben ser mayores que cero.

Los resultados obtenidos en la terminal para estas pruebas se muestran en la Figura 10.

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65535 2
65535;2;1.07371e+09;1.07371e+09;1.07371e+09;2073397;123320;84;
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65535 3
65535;3;7.15839e+08;7.15839e+08;7.15839e+08;2464167;158802;98;
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65535 10
65535;10;2.14742e+08;2.14742e+08;2.14742e+08;2301439;161810;153;
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 2
Error: n y k deben ser mayores que cero.
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 3
Error: n y k deben ser mayores que cero.
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 10
Error: n y k deben ser mayores que cero.
isa21067@LAPTOP-RR7NMTBU: /mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$

```

Fig. 10. Pruebas realizadas para $n = 65535$ y $n = 65536$ con $k = 2$, $k = 3$ y $k = 10$. Fuente: elaboración propia.

Inicialmente no se conocía la razón por la cual el programa aceptaba 65535 pero no 65536, por lo que se revisó el tipo de dato utilizado para almacenar n . Se encontró que el programa utilizaba `unsigned short`.

De acuerdo con la información consultada sobre los tipos fundamentales de C++, un entero sin signo de 16 bits puede representar valores desde 0 hasta 65535 [12]. Por esta razón, 65535 todavía puede almacenarse dentro de este rango, mientras que 65536 ya lo supera.

Al ingresar 65536, el valor que terminaba almacenado en n era 0. Posteriormente, este valor era detectado por la condición:

```
if (n == 0 || k == 0)
```

Al cumplirse esta condición, el programa mostraba el mensaje de error y terminaba antes de ejecutar las funciones de suma. Esto también permitió observar que el problema no dependía del valor de k , ya que se presentó el mismo comportamiento utilizando $k = 2$, $k = 3$ y $k = 10$.

Modificaciones realizadas Para corregir esta limitación se modificaron los tipos de datos utilizados en diferentes partes del programa.

En el archivo `suma_functions.h` se cambiaron los parámetros n y k de las declaraciones de `suma_recursive`, `suma_iterative` y `suma_constant`, reemplazando `unsigned short` por `unsigned int`.

Las declaraciones quedaron de la siguiente manera:

```
double suma_recursive(unsigned int n, unsigned int k);
double suma_iterative(unsigned int n, unsigned int k);
double suma_constant(unsigned int n, unsigned int k);
```

El mismo cambio se realizó en `suma_functions.cxx`, modificando los parámetros de las definiciones de las tres funciones para que coincidieran con las declaraciones del archivo de encabezado.

Además, en la función `suma_iterative` se cambió la variable utilizada para acumular la suma. Inicialmente se utilizaba:

```
int suma = 0;
```

y se reemplazó por:

```
unsigned long long suma = 0;
```

Este cambio se realizó porque, al permitir valores mayores de n , el resultado acumulado también puede aumentar considerablemente y podría superar el rango disponible para un `int`.

En la función `suma_constant` también se modificó el tipo utilizado para almacenar la cantidad de múltiplos. La variable m se definió como:

```
unsigned long long m = n / k;
```

De esta manera, se dispone de un rango mayor para las operaciones intermedias realizadas por la fórmula.

Finalmente, en `test_suma.cxx` se modificaron las variables que reciben los argumentos ingresados desde la terminal. Originalmente, n y k eran almacenados como `unsigned short`. Después del cambio quedaron de la siguiente manera:

```
unsigned int n = static_cast<unsigned int>(std::atoi(argv[1]));
unsigned int k = static_cast<unsigned int>(std::atoi(argv[2]));
```

La condición que verifica que n y k sean mayores que cero no fue modificada, ya que continúa siendo válida. El problema se encontraba en que el valor 65536 no podía representarse mediante el tipo utilizado anteriormente.

Pruebas después de la modificación Después de realizar los cambios, se recompiló el programa mediante el comando:

```
g++ -std=c++17 suma_functions.cxx test_suma.cxx -o my_test_suma
```

Luego se repitieron las pruebas con $n = 65536$ para los mismos valores de k .

Para $k = 2$, las tres versiones produjeron 1.07377×10^9 . Para $k = 3$, las tres obtuvieron 7.15839×10^8 y, para $k = 10$, las tres obtuvieron 2.14742×10^8 .

Por lo tanto, después de las modificaciones, $n = 65536$ pudo ser procesado correctamente y las versiones recursiva, iterativa y constante volvieron a producir resultados numéricos iguales.

La recompilación y las pruebas realizadas después de modificar el código se muestran en la Figura 11.

A partir de estos resultados se concluye que el error encontrado no estaba relacionado directamente con la lógica de las tres formas de calcular la suma, sino con el rango del tipo de dato utilizado para recibir la entrada.

Cambiar `unsigned short` por `unsigned int` permitió ampliar el rango de n , mientras que el uso de `unsigned long long` en las operaciones que pueden producir valores mayores permite disponer de un rango más amplio para los resultados intermedios.

```
isa21087@LAPTOP-RK7NHTBU:/mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ g++ -std=c++17 suma_functions.cpp test_suma.cpp -o my_test_s
uma
isa21087@LAPTOP-RK7NHTBU:/mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 2
65536;2;1.87377e+09;1.87377e+09;1.87377e+09;2311553;15792876;
isa21087@LAPTOP-RK7NHTBU:/mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 3
65536;3;7.15839e+08;7.15839e+08;7.15839e+08;2461799;21316659;
isa21087@LAPTOP-RK7NHTBU:/mnt/c/Users/Usuario/Desktop/tallerescorte/taller_1_complejidad_archivos$ ./my_test_suma 65536 10
65536;10;2.14742e+08;2.14742e+08;2.14742e+08;1346273;80809144;
```

Fig. 11. Recompilación y pruebas con $n = 65536$ después de modificar los tipos de datos utilizados por el programa. Fuente: elaboración propia.

2.4 Relación con la eficiencia teórica

Además del análisis empírico realizado anteriormente, se analizó la complejidad teórica de cada una de las versiones del algoritmo. Para ello se tuvo en cuenta la cantidad de operaciones realizadas dependiendo del tamaño de entrada n .

En el material de clase, este análisis se realiza contando las operaciones primitivas de cada parte del código, construyendo una función $T(n)$ y posteriormente simplificándola para determinar su comportamiento asintótico [13].

Versión recursiva En la versión recursiva, la función disminuye el valor de n en uno cada vez que realiza una nueva llamada, hasta alcanzar el caso base. Esto hace que el número de llamadas aumente a medida que aumenta el valor de entrada.

Para entender mejor este comportamiento, se realizó el conteo de las operaciones directamente sobre el código, como se muestra en la Figura 12.

```
// ANTES double suma_recursive(unsigned short n, unsigned short k) {
double suma_recursive(unsigned int n, unsigned int k) {
    if (n == 0) {                                // 1
        return 0;                                // 1
    }

    if (n % k == 0) {                             // 2
        return n + suma_recursive(n - 1, k);      // 2 + T(n-1)
    }
    else {
        return suma_recursive(n - 1, k);          // 1 + T(n-1)
    }
}
//T(n) = 1 + 2 + 2 + T(n-1)
//T(n) = 5 + T(n-1)
```

Fig. 12. Conteo de operaciones de la versión recursiva. Fuente: elaboración propia.

Tomando el caso en el que se realiza la mayor cantidad de operaciones en cada llamada, se obtiene la siguiente recurrencia:

$$T(n) = 1 + 2 + 2 + T(n - 1)$$

Por lo tanto,

$$T(n) = 5 + T(n - 1)$$

Para observar cómo crece esta expresión, se puede desarrollar la recurrencia:

$$T(n) = 5 + 5 + T(n - 2)$$

$$T(n) = 5 + 5 + 5 + T(n - 3)$$

Si se continúa sustituyendo de la misma manera hasta llegar al caso base, se obtiene:

$$T(n) = 5n + T(0)$$

En este resultado, el término que depende directamente del tamaño de entrada es $5n$. Al eliminar las constantes y conservar el término de mayor crecimiento, la complejidad corresponde a:

$$T(n) = O(n)$$

Por lo tanto, la versión recursiva presenta una complejidad lineal. Esto se debe a que, a medida que aumenta n , también aumenta proporcionalmente la cantidad de llamadas recursivas necesarias para llegar al caso base.

Versión iterativa La versión iterativa utiliza un ciclo que recorre los valores hasta llegar a n . Durante cada iteración se verifica si el valor actual es múltiplo de k y, cuando se cumple esta condición, se agrega al acumulador.

El conteo de operaciones realizado sobre esta versión se muestra en la Figura 13.

```
//double suma_iterative(unsigned short n, unsigned short k) {
double suma_iterative(unsigned int n, unsigned int k) {
    //ANTES int suma = 0;
    unsigned long long suma = 0;           // 1

    for (unsigned int i = 1; i <= n; i++)    // 1 + (n+1) + n
    {
        if (i % k == 0)                    // 2*n
        {
            suma += i;                     // 2*n
        }
    }

    return suma;                           // 1
}
//T(n) = 1 + [1 + (n+1) + n] + 2n + 2n + 1
```

Fig. 13. Conteo de operaciones de la versión iterativa. Fuente: elaboración propia.

A partir del conteo realizado se obtiene:

$$T(n) = 1 + [1 + (n + 1) + n] + 2n + 2n + 1$$

Al agrupar los términos:

$$T(n) = 1 + 1 + n + 1 + n + 2n + 2n + 1$$

se obtiene:

$$T(n) = 6n + 4$$

En este caso, el término que presenta el mayor crecimiento es $6n$. Al eliminar las constantes y los factores constantes, se obtiene:

$$T(n) = O(n)$$

Por lo tanto, la versión iterativa también presenta una complejidad lineal, ya que la cantidad de iteraciones aumenta junto con el tamaño de entrada.

Aunque la versión recursiva y la iterativa tienen la misma complejidad $O(n)$, esto no significa que presenten exactamente los mismos tiempos de ejecución. La notación Big-O permite describir cómo crece la cantidad de operaciones cuando aumenta el tamaño de entrada, pero no representa directamente el tiempo medido durante una ejecución real [13].

Versión constante La versión constante presenta un funcionamiento diferente, ya que no utiliza ciclos ni llamadas recursivas. En lugar de recorrer los valores hasta n , aplica directamente la fórmula desarrollada anteriormente.

El conteo de operaciones de esta función se muestra en la Figura 14.

```
//ANTES double suma_constant(unsigned short n, unsigned short k) {
double suma_constant(unsigned int n, unsigned int k) {
    // TODO #01: Implementar esta función utilizando una fórmula matemática
    // para calcular directamente la suma de los múltiplos de k menores o iguales a n.
    unsigned long long m = n / k;           // 2
    return k * m * (m + 1) / 2;             // 5
}
//T(n) = 2 + 5
//T(n) = 7
//T(n) = O(1)
// eof - suma_functions.cxx
```

Fig. 14. Conteo de operaciones de la versión constante. Fuente: elaboración propia.

De acuerdo con el conteo realizado:

$$T(n) = 2 + 5$$

Por lo tanto:

$$T(n) = 7$$

A diferencia de las versiones anteriores, esta expresión no contiene ningún término que dependa de n . Esto significa que aumentar el tamaño de la entrada no aumenta la cantidad de operaciones principales necesarias para calcular el resultado.

Por esta razón, su complejidad es:

$$T(n) = O(1)$$

La versión constante presenta entonces una complejidad constante, ya que realiza una cantidad fija de operaciones independientemente del tamaño de entrada.

Comparación de las tres versiones Los resultados obtenidos mediante el análisis teórico se resumen en la Tabla 1.

Table 1. Complejidad temporal de las tres versiones del algoritmo.

Versión	Complejidad temporal
Recursiva	$O(n)$
Iterativa	$O(n)$
Constante	$O(1)$

Al comparar estos resultados con las gráficas obtenidas anteriormente, se observa una relación entre el comportamiento teórico y el empírico.

La versión recursiva presentó los tiempos de ejecución más altos y una tendencia general de crecimiento conforme aumentaba n . La versión iterativa también mostró un crecimiento en sus tiempos, pero estos se mantuvieron considerablemente por debajo de los obtenidos por la versión recursiva.

Aunque ambas presentan una complejidad $O(n)$, esto no significa que deban tener exactamente el mismo tiempo de ejecución. Dos algoritmos pueden pertenecer al mismo orden de complejidad y aun así realizar una cantidad diferente de operaciones o tener costos adicionales asociados a su implementación.

En este caso, la versión recursiva necesita realizar llamadas sucesivas a la función, mientras que la versión iterativa utiliza un ciclo. Por esta razón, aun cuando ambas crecen linealmente respecto a n , sus tiempos medidos pueden ser diferentes.

Por otro lado, la versión constante presentó los tiempos más bajos y relativamente estables para $k = 2$, $k = 3$ y $k = 10$. Al observar las gráficas completas, su curva se encontraba prácticamente sobre el eje horizontal debido a la diferencia de escala con la versión recursiva.

Al analizar únicamente los datos de la versión constante también se observaron algunas variaciones y picos aislados. Sin embargo, estos valores no muestran una tendencia de crecimiento sostenida conforme aumenta n .

Esto concuerda con la complejidad $O(1)$ obtenida teóricamente. Que la función tenga complejidad constante no significa que todas las mediciones deban producir exactamente el mismo número de nanosegundos, sino que la cantidad de operaciones del algoritmo no aumenta en función del tamaño de entrada [13].

En conjunto, los resultados empíricos concuerdan con las complejidades teóricas obtenidas. Las versiones recursiva e iterativa muestran un crecimiento relacionado con

el aumento de n , de acuerdo con su complejidad $O(n)$, mientras que la versión constante no presenta un crecimiento sostenido con el tamaño de entrada, lo cual es coherente con $O(1)$.

2.5 Reflexión final

A partir de los análisis teóricos y empíricos realizados, se considera que la versión constante es la más adecuada para utilizar en un programa real. En las pruebas realizadas, esta versión produjo los mismos resultados que las versiones recursiva e iterativa, pero presentó tiempos de ejecución considerablemente menores y una complejidad teórica $O(1)$.

Esto se debe a que utiliza directamente la fórmula desarrollada para obtener el resultado, sin necesidad de recorrer todos los valores hasta n ni realizar llamadas recursivas.

Por otro lado, aunque las versiones recursiva e iterativa presentan ambas una complejidad $O(n)$, sus comportamientos empíricos fueron diferentes. La versión recursiva presentó los tiempos de ejecución más altos en las gráficas, mientras que la iterativa se mantuvo considerablemente por debajo.

Para este problema en particular, la recursión no aporta una ventaja que justifique ese mayor tiempo de ejecución, por lo que no sería la primera opción para una implementación real.

La recomendación también depende del rango de valores que se espera manejar. Como se observó anteriormente en la Figura 10, con $n = 65535$ el programa funcionaba correctamente, mientras que con $n = 65536$ se generaba un error debido al rango del tipo de dato utilizado.

Después de realizar las modificaciones en los tipos de datos, la Figura 11 muestra que $n = 65536$ pudo ser procesado correctamente para $k = 2$, $k = 3$ y $k = 10$.

Este resultado permitió observar que la elección de un algoritmo no depende únicamente de su complejidad temporal. También es necesario considerar el rango de valores que pueden representar los tipos de datos utilizados, ya que incluso una solución eficiente puede presentar errores si las entradas o los resultados superan dichos límites.

Por esta razón, para este problema se recomienda utilizar la versión constante junto con tipos de datos adecuados para el rango esperado de entrada y de resultados.

Para valores pequeños, las tres versiones pueden producir resultados correctos, pero a medida que n aumenta, la versión constante ofrece una mayor ventaja debido a que su cantidad de operaciones no crece con el tamaño de la entrada.

Si el programa tuviera que trabajar con valores todavía mayores, además de mantener la versión constante sería necesario revisar nuevamente los tipos de datos utilizados para evitar posibles problemas de desbordamiento.

3 Sort

3.1 Análisis empírico inicial

Para analizar el comportamiento de los algoritmos de ordenamiento se compararon los tiempos de ejecución de BubbleSort, QuickSort y HeapSort utilizando tres tipos de

entrada: aleatoria, ordenada e inversa. En todos los casos se utilizó el tamaño del arreglo n como eje horizontal y el tiempo de ejecución en milisegundos como eje vertical.

Entrada aleatoria En la entrada aleatoria se observa que BubbleSort presenta el crecimiento más pronunciado. Para valores pequeños de n , los tiempos todavía son bajos, pero a medida que aumenta el tamaño del arreglo la curva empieza a crecer de forma considerable y presenta varias variaciones y picos.

Hacia los valores más altos de n , BubbleSort supera ampliamente a los otros dos algoritmos y llega a valores cercanos a 1400 ms.

Por otro lado, QuickSort y HeapSort se mantienen prácticamente sobre el eje horizontal en comparación con BubbleSort. Esto se debe a que sus tiempos son mucho menores dentro de la escala utilizada en la gráfica. En esta entrada, ambos presentan un comportamiento bastante estable y no muestran un crecimiento tan marcado como BubbleSort.

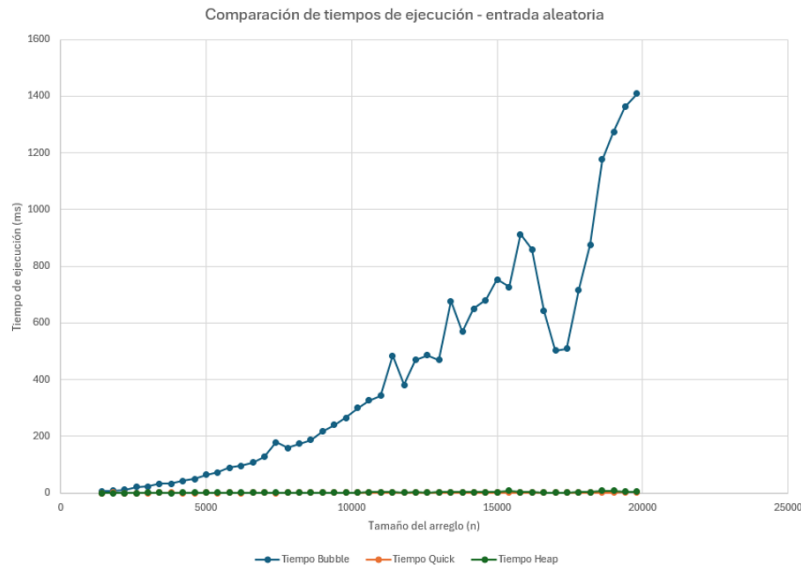


Fig. 15. Comparación de tiempos de ejecución para entrada aleatoria. Fuente: elaboración propia.

De acuerdo con los tiempos observados, QuickSort y HeapSort presentan el mejor desempeño para esta entrada, con valores considerablemente menores que BubbleSort.

Entrada ordenada En la entrada ordenada se observa un comportamiento diferente. BubbleSort y QuickSort presentan tiempos mucho más cercanos entre sí y sus curvas incluso se cruzan en distintos puntos. A medida que aumenta n , ambos algoritmos muestran una tendencia general creciente, aunque acompañada de varias fluctuaciones.

Para algunos tamaños de entrada, QuickSort supera a BubbleSort, mientras que en otros ocurre lo contrario. Hacia los valores más grandes del arreglo, ambos se encuentran aproximadamente entre 700 ms y 800 ms. Por esta razón, en esta entrada no se observa una diferencia tan marcada entre los dos algoritmos como la encontrada anteriormente con los datos aleatorios.

HeapSort, en cambio, permanece nuevamente muy cerca del eje horizontal durante toda la gráfica. Sus tiempos son mucho menores que los de BubbleSort y QuickSort, incluso cuando aumenta el tamaño del arreglo.

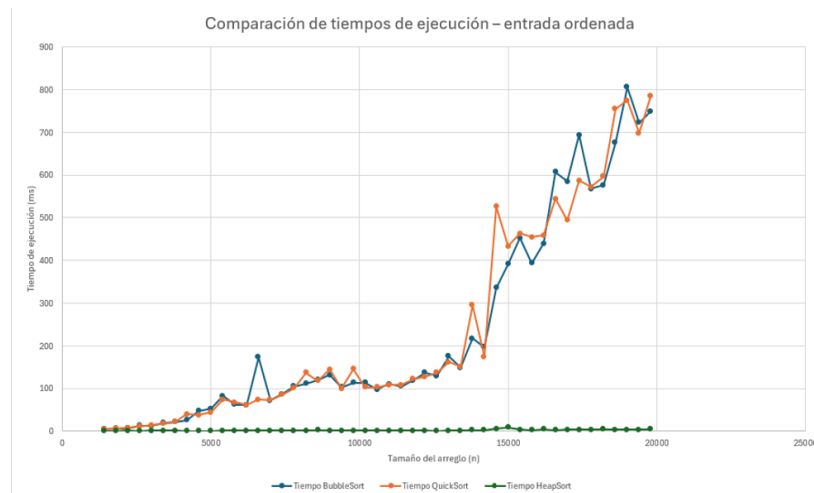


Fig. 16. Comparación de tiempos de ejecución para entrada ordenada. Fuente: elaboración propia.

En esta configuración, HeapSort presenta el mejor desempeño observado, ya que mantiene los tiempos más bajos durante toda la prueba.

Entrada inversa Para la entrada inversa, BubbleSort vuelve a presentar los mayores tiempos de ejecución. Su curva muestra una tendencia creciente bastante clara y alcanza aproximadamente 1300 ms para los valores más altos de n .

Aunque existen algunos descensos y picos intermedios, el comportamiento general sigue mostrando un aumento del tiempo conforme crece el tamaño del arreglo.

QuickSort también presenta un aumento considerable en esta entrada y alcanza tiempos superiores a 500 ms para los arreglos más grandes. Sin embargo, se mantiene por debajo de BubbleSort durante prácticamente toda la prueba.

HeapSort continúa presentando los tiempos más bajos. Su curva permanece casi sobre el eje horizontal en comparación con las otras dos, mostrando muy poca variación incluso al aumentar n .

Para la entrada inversa, HeapSort vuelve a presentar el mejor desempeño, seguido por QuickSort y finalmente BubbleSort.

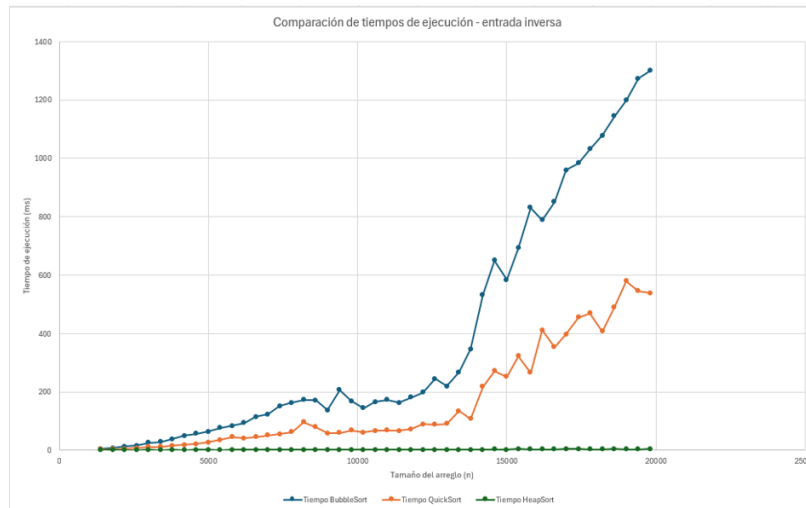


Fig. 17. Comparación de tiempos de ejecución para entrada inversa. Fuente: elaboración propia.

Comparación general Al comparar las tres entradas, HeapSort presenta el comportamiento más estable y los menores tiempos de ejecución en las pruebas realizadas. BubbleSort, por el contrario, muestra los mayores tiempos en las entradas aleatoria e inversa y es el algoritmo que presenta el crecimiento más marcado.

QuickSort cambia más dependiendo del tipo de entrada. Con datos aleatorios sus tiempos son muy bajos y cercanos a los de HeapSort, mientras que con entradas ordenadas e inversas sus tiempos aumentan de manera considerable. En la entrada ordenada incluso llega a presentar un comportamiento muy similar al de BubbleSort.

3.2 Complejidad teórica: mejor, promedio y peor caso

Para comparar los tres algoritmos de ordenamiento, se revisaron sus complejidades teóricas en el mejor caso, caso promedio y peor caso. Los resultados se resumen en la Tabla 2 [14,15,16].

Table 2. Complejidad teórica de los algoritmos de ordenamiento.

Algoritmo	Mejor caso	Caso promedio	Peor caso
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

BubbleSort BubbleSort presenta un comportamiento cuadrático en el caso promedio y en el peor caso. Esto ocurre porque realiza varios recorridos sobre el arreglo y en cada uno compara elementos adyacentes para intercambiarlos cuando están desordenados.

A medida que aumenta n , la cantidad total de comparaciones también aumenta. En el peor caso se realizan aproximadamente $\frac{1}{2}n^2$ comparaciones e intercambios [16]. Por esta razón, para estos casos se obtiene:

$$O(n^2)$$

En una implementación optimizada de BubbleSort que incluya una condición de parada temprana, el mejor caso puede alcanzar:

$$O(n)$$

Esto ocurre porque, cuando el arreglo ya se encuentra ordenado, el algoritmo puede realizar un recorrido, detectar que no hubo intercambios y terminar sin ejecutar las pasadas restantes [16].

Sin embargo, la implementación utilizada en este taller no contiene esta optimización. Por esta razón, el mejor caso lineal de la versión optimizada no necesariamente se refleja en las pruebas realizadas con el arreglo ordenado.

QuickSort QuickSort presenta una complejidad de:

$$O(n \log n)$$

en el mejor caso y en el caso promedio [15].

El término $\log n$ aparece porque, cuando las particiones son relativamente equilibradas, el problema se va dividiendo en subarreglos cada vez más pequeños. Si aproximadamente se reduce a la mitad en cada nivel, se generan cerca de $\log n$ niveles de división.

Como en cada nivel se procesan aproximadamente n elementos, el crecimiento total puede representarse como:

$$n \log n$$

En cambio, en el peor caso QuickSort puede llegar a:

$$O(n^2)$$

Esto sucede cuando el pivote genera particiones muy desbalanceadas, dejando muy pocos elementos en una parte y casi todos los demás en la otra. En ese caso, el problema deja de reducirse aproximadamente a la mitad y puede disminuir casi de uno en uno [15].

HeapSort HeapSort mantiene una complejidad de:

$$O(n \log n)$$

en el mejor caso, caso promedio y peor caso [14].

Este comportamiento se relaciona con la estructura *heap*. La altura de un heap crece aproximadamente de forma logarítmica respecto a n , por lo que las operaciones necesarias para reorganizarlo tienen un costo relacionado con $\log n$.

Como estas operaciones se realizan sobre los elementos del arreglo, el comportamiento total corresponde a $n \log n$.

A diferencia de QuickSort, HeapSort mantiene el mismo orden de crecimiento en los tres casos. Por esta razón, teóricamente presenta un comportamiento más estable frente a diferentes formas de organización inicial de los datos.

Comparación general BubbleSort presenta un crecimiento cuadrático en el caso promedio y en el peor caso. QuickSort normalmente presenta un comportamiento $O(n \log n)$, aunque puede llegar a $O(n^2)$ cuando las particiones quedan muy desbalanceadas. HeapSort, por su parte, mantiene $O(n \log n)$ en los tres escenarios.

3.3 Relación entre teoría y observación empírica

Después de revisar las complejidades teóricas de BubbleSort, QuickSort y HeapSort, se compararon estos resultados con las gráficas obtenidas para las entradas aleatoria, ordenada e inversa.

En algunos casos el comportamiento observado fue similar a lo esperado, mientras que en otros fue necesario revisar la implementación para entender mejor por qué se presentaban ciertas diferencias.

Entrada aleatoria La entrada aleatoria puede relacionarse principalmente con el caso promedio de los algoritmos. En la gráfica correspondiente se observa que BubbleSort presenta un crecimiento mucho mayor que QuickSort y HeapSort. A medida que aumenta el tamaño del arreglo, sus tiempos también aumentan considerablemente, llegando a superar los 1400 ms para los valores más altos de n .

Este comportamiento se relaciona con su complejidad promedio $O(n^2)$, ya que BubbleSort realiza varios recorridos sobre el arreglo y una gran cantidad de comparaciones entre elementos.

QuickSort, en cambio, permanece con tiempos muy bajos para este tipo de entrada. Esto coincide con su caso promedio $O(n \log n)$. Con datos aleatorios, el pivote tiene una mayor posibilidad de generar particiones relativamente equilibradas, por lo que el arreglo puede dividirse en partes más pequeñas sin que una sola partición concentre casi todos los elementos [15].

HeapSort también mantiene tiempos muy bajos durante toda la prueba. Esto coincide con su complejidad $O(n \log n)$, la cual mantiene el mismo orden de crecimiento para los diferentes tipos de entrada analizados [14].

Por lo tanto, para la entrada aleatoria, el comportamiento observado es cercano a lo esperado teóricamente: BubbleSort aumenta mucho más rápido, mientras que QuickSort y HeapSort presentan tiempos considerablemente menores.

Entrada ordenada En la entrada ordenada se observó un comportamiento diferente al de la entrada aleatoria. BubbleSort y QuickSort presentan tiempos mucho más cercanos entre sí y ambos aumentan considerablemente a medida que crece n . HeapSort, por otro lado, continúa muy cerca del eje horizontal.

En el caso de BubbleSort, inicialmente podría esperarse un mejor comportamiento debido a que el arreglo ya se encuentra ordenado. Teóricamente, este escenario puede representar un mejor caso $O(n)$ cuando el algoritmo cuenta con una condición de parada temprana.

Sin embargo, al revisar la implementación utilizada en el taller se observa que BubbleSort contiene dos ciclos que continúan ejecutándose incluso cuando no es necesario realizar intercambios. No existe una condición que permita detectar que el arreglo ya está ordenado y terminar el proceso.

La implementación utilizada se muestra en la Figura 18.

```
//
void bubblesort(long* a, long n)
{
    for (unsigned long j = 0; j < n; ++j)
    {
        for (unsigned long i = 0; i < n - 1; ++i)
        {
            if (a[i] > a[i + 1])
            {
                long tmp = a[i];
                a[i] = a[i + 1];
                a[i + 1] = tmp;
            } // fi
        } // rof
    } // rof
}
```

Fig. 18. Implementación de BubbleSort utilizada en el taller. Fuente: código proporcionado para el taller.

Esto permite entender por qué en la gráfica la entrada ordenada no presenta el mejor comportamiento lineal que normalmente puede asociarse con una versión optimizada de BubbleSort. Aunque los elementos ya estén en su posición correcta, las comparaciones continúan realizándose debido a los ciclos presentes en la implementación.

QuickSort también presenta un comportamiento bastante diferente al observado con la entrada aleatoria. Sus tiempos aumentan considerablemente y en varios puntos llegan a ser similares a los de BubbleSort.

Para entender esta diferencia se revisó la función de partición. Allí se observa que el pivote siempre corresponde al último elemento del arreglo:

$$x = a[r]$$

Esta parte del código se muestra en la Figura 19.

```
// -----
long quicksort_partition(long* a, long p, long r)
{
    long x = a[r];    // Pivote //toma como pivote el último elemento del arreglo
    long j = p - 1;    // índice del último elemento menor o igual al pivote

    for (long i = p; i < r; i++)
    {
        if (x >= a[i])
        {
            j = j + 1;

            // Intercambiar a[i] con a[j]
            long temp = a[j];
            a[j] = a[i];
            a[i] = temp;
        } // fi
    } // rof

    // Colocar el pivote en su posición final
    a[r] = a[j + 1];
    a[j + 1] = x;

    return (j + 1);
}

// -----
// Función recursiva auxiliar para quicksort.
// Aplica recursivamente el algoritmo en los subarreglos izquierdo y derecho
// delimitados por el índice de partición.
// -----
void quicksort_dummy(long* a, long p, long r)
{
    if (p < r)
    {
        long q = quicksort_partition(a, p, r);    // Partición
        quicksort_dummy(a, p, q - 1);            // Subarreglo izquierdo
    }
}
```

Fig. 19. Selección del pivote e implementación de QuickSort utilizada en el taller. Fuente: código proporcionado para el taller.

Cuando el arreglo ya se encuentra ordenado, el último elemento corresponde a uno de los extremos de los valores del arreglo. Al utilizarlo como pivote, las particiones pueden quedar muy desbalanceadas: una parte queda con muy pocos elementos y la otra conserva prácticamente todos los restantes.

En lugar de reducir el problema aproximadamente a la mitad en cada llamada, el tamaño puede disminuir casi de uno en uno. Esto permite relacionar el comportamiento observado con el peor caso de QuickSort:

$$O(n^2)$$

La forma en la que se selecciona el pivote es importante para el rendimiento de QuickSort, especialmente cuando los datos ya presentan cierto orden [15].

Entrada inversa Para la entrada inversa, BubbleSort vuelve a presentar los mayores tiempos de ejecución. En este caso los elementos se encuentran en el orden contrario al esperado, por lo que el algoritmo debe realizar una gran cantidad de comparaciones e intercambios antes de completar el ordenamiento.

Por esta razón, este escenario puede relacionarse con uno de los casos más desfavorables para BubbleSort y su comportamiento $O(n^2)$. Esto coincide con la gráfica obtenida, donde su tiempo aumenta de manera marcada conforme crece n .

QuickSort también presenta un aumento considerable frente a la entrada aleatoria. Al revisar nuevamente la forma en la que se selecciona el pivote, utilizar siempre el último elemento también puede generar particiones muy desbalanceadas cuando el arreglo se encuentra en orden inverso.

En este caso, el pivote vuelve a encontrarse en uno de los extremos de los valores disponibles. Esto explica que sus tiempos sean mayores que en la entrada aleatoria y que su comportamiento pueda acercarse nuevamente al peor caso $O(n^2)$ [15].

Aun así, en las pruebas realizadas QuickSort se mantiene por debajo de BubbleSort durante prácticamente toda la gráfica.

HeapSort vuelve a presentar el comportamiento más estable. Al revisar su implementación se observa que el algoritmo utiliza las operaciones `make_heap` y `sort_heap`, como se muestra en la Figura 20.

```
// -----
// Implementación de Heapsort utilizando funciones de la STL.
// make_heap construye un heap (montículo máximo).
// sort_heap ordena el arreglo a partir del heap generado.
// -----
void heapsort(long* a, long n)
{
    std::make_heap(a, a + n); // Construye el heap
    std::sort_heap(a, a + n); // Ordena el arreglo a partir del heap
}

// eof - sort_functions.cxx
```

Fig. 20. Implementación de HeapSort utilizada en el taller. Fuente: código proporcionado para el taller.

HeapSort mantiene una complejidad $O(n \log n)$ en el mejor caso, caso promedio y peor caso [14]. Por esta razón, el orden inicial de los elementos no modifica su orden de crecimiento teórico de la misma forma que ocurre con QuickSort.

Esto también se observa en las gráficas, ya que sus tiempos permanecen muy bajos para las tres configuraciones de entrada.

Comparación general Al comparar las tres gráficas con la teoría, se observa que los resultados en general son coherentes con las complejidades estudiadas, pero también muestran la importancia de revisar la implementación concreta de cada algoritmo.

BubbleSort presenta los mayores tiempos en las entradas aleatoria e inversa y no obtiene la mejora esperada con la entrada ordenada. Al revisar el código se entiende que esto ocurre porque la implementación no cuenta con una condición de parada temprana.

QuickSort es el algoritmo donde más se nota la influencia del tipo de entrada. Con datos aleatorios presenta tiempos muy bajos, mientras que con datos ordenados e inversos aumenta considerablemente. La revisión de la función de partición permite relacionar este resultado con la selección del último elemento como pivote, la cual puede generar particiones muy desbalanceadas.

HeapSort presenta el comportamiento más estable de los tres. Sus tiempos se mantienen bajos en las entradas aleatoria, ordenada e inversa, lo cual coincide con su complejidad $O(n \log n)$ en los tres escenarios.

También aparecen algunos picos y variaciones en las mediciones. Estos valores no necesariamente representan un cambio en la complejidad del algoritmo, ya que los tiempos empíricos pueden verse afectados por factores propios de la ejecución, como el uso de memoria, la caché, otros procesos activos en el sistema o pequeñas variaciones entre una ejecución y otra.

En general, las gráficas presentan un comportamiento coherente con la teoría. Sin embargo, el análisis también muestra que no basta con conocer únicamente la complejidad general de un algoritmo, ya que la forma específica en la que está implementado puede modificar considerablemente su comportamiento frente a determinados tipos de entrada.

4 Uso de herramientas de inteligencia artificial generativa

Durante el desarrollo del taller se utilizó ChatGPT como herramienta de apoyo en distintas partes del proceso. Su uso se concentró principalmente en la revisión y organización de la redacción, la aclaración de conceptos de complejidad algorítmica, la explicación de fragmentos del código proporcionado en el taller y el apoyo para interpretar los resultados obtenidos en las pruebas.

En particular, se utilizó apoyo de inteligencia artificial durante la redacción y revisión de los análisis presentados en las secciones correspondientes a la suma de múltiplos y a los algoritmos de ordenamiento, especialmente para comprender la relación entre los resultados empíricos y la complejidad teórica. También se utilizó para comprender el comportamiento del programa con $n = 65535$ y $n = 65536$, y para analizar características específicas de las implementaciones de BubbleSort y QuickSort.

Las respuestas obtenidas mediante la herramienta no se incorporaron directamente sin revisión. Se adaptó la redacción para conservar las ideas y resultados obtenidos durante el desarrollo del taller, se eliminaron explicaciones que no correspondían con las pruebas realizadas, se contrastaron los conceptos con las fuentes bibliográficas utilizadas y se relacionaron las explicaciones únicamente con los resultados observados en las ejecuciones, capturas y gráficas propias. Las ejecuciones de los programas, modificaciones realizadas al código, capturas, tablas, gráficas y resultados numéricos fueron obtenidos durante el desarrollo del taller.

A continuación se presentan algunos de los prompts utilizados durante el desarrollo del taller.

Apoyo en la redacción

Te voy a pasar un texto que yo escribí con mis propias ideas y resultados del taller. Ayúdame a organizarlo y redactarlo mejor, pero sin cambiar lo que quiero decir ni inventar información. Quiero que conserve un lenguaje estudiantil, natural y claro, no demasiado técnico ni formal. Corrige ortografía, repeticiones y organización de las ideas. Si hace falta complementar algo para que se entienda, hazlo solo con información que esté sustentada por las fuentes que te pase. También quiero que mantengas las citas dentro del texto.

Aclaración de conceptos

Estoy haciendo un taller de complejidad algorítmica y hay conceptos que todavía no manejo bien porque estoy empezando con estructuras de datos. Te voy a pasar el material o la parte que no entiendo. Explícamela paso a paso y con palabras sencillas, sin asumir que ya conozco el tema. No quiero solamente la respuesta, quiero entender por qué funciona así para después poder explicarlo con mis propias palabras en el informe. Si utilizas algún término nuevo, explícame primero qué significa.

Comprensión de $O(n \log n)$

En una fuente que estoy utilizando para investigar QuickSort y HeapSort aparece que algunos casos tienen complejidad $O(n \log n)$. Ya leí la referencia, pero no entiendo de dónde sale el logaritmo ni por qué se multiplica por n . Explícame paso a paso qué significa $O(n \log n)$, de dónde aparece el $\log n$ cuando un problema se va dividiendo y por qué eso se relaciona con estos algoritmos. Quiero entenderlo de manera sencilla para después poder relacionarlo con la información de la fuente y redactarlo con mis propias palabras.

Explicación de código línea por línea

Estoy empezando a ver estructuras de datos y todavía hay partes de este código en C++ que no conozco. Te voy a pegar una función del taller. Explícame línea por línea qué hace, qué significa cada variable, cada condición, cada ciclo y cada llamada a otra función. También explícame cuál es la lógica general del algoritmo y cómo se conectan las líneas entre sí. No modifiques el código ni me des directamente una solución diferente, primero ayúdame a entender el código que ya tengo.

Análisis de QuickSort

Explícame paso a paso el código de QuickSort que me dieron en el taller. No conozco bien todavía cómo funciona la partición ni el pivote. Quiero entender qué hace la función de partición, qué significa que se utilice el último elemento como pivote y qué pasa con las llamadas recursivas. Después ayúdame a relacionar esa implementación con los resultados que obtuve cuando el arreglo estaba aleatorio, ordenado e inverso.

Análisis de BubbleSort

Estoy comparando la complejidad teórica de BubbleSort con la implementación que me dieron. En las fuentes aparece que el mejor caso puede ser $O(n)$, pero en mis resultados con el arreglo ordenado no parece comportarse así. Ayúdame a revisar el código y explícame si tiene alguna condición para detenerse cuando el arreglo ya está ordenado. Explícame por qué tener o no esa condición puede cambiar el mejor caso.

Análisis del error con 65535 y 65536

Ejecuté el programa con $n = 65535$ y funciona, pero con $n = 65536$ aparece un error diciendo que n y k deben ser mayores que cero. No entiendo por qué pasa si 65536 sí es mayor que cero. Ayúdame a revisar el código paso a paso para identificar qué tipo de dato se está utilizando, cuál es su rango y por qué el valor cambia. Después explícame qué modificación podría hacerse para permitir valores mayores y qué partes del código habría que cambiar, porque leí su documentación y hay varios tipos de datos.

Referencias

1. Arangoya Centro Educativo, *Programación modular: cómo mejora el desarrollo de software*. Disponible en: <https://arangoya.org/programacion-modular-como-mejora-el-desarrollo-de-software/>. Consultado el 4 de agosto de 2026.
2. Microsoft, *Header Files (C++)*. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/cpp/cpp/header-files-cpp>. Consultado el 4 de agosto de 2026.
3. OpenAI, *ChatGPT*. Herramienta utilizada como apoyo para aclarar conceptos, comprender fragmentos de código y organizar y revisar la redacción del informe. Consultado en agosto de 2026.
4. Free Software Foundation, *Overall Options — Using the GNU Compiler Collection*. GCC Documentation. Disponible en: <https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html>. Consultado el 5 de agosto de 2026.
5. Microsoft, *Source Code Organization (C++ Templates)*. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/cpp/cpp/source-code-organization-cpp-templates>. Consultado el 4 de agosto de 2026.
6. cppreference.com, *Phases of translation*. C++ Language. Disponible en: https://en.cppreference.com/w/cpp/language/translation_phases. Consultado el 4 de agosto de 2026.
7. Free Software Foundation, *Bash Reference Manual*. GNU Project. Disponible en: <https://www.gnu.org/software/bash/manual/bash.html>. Consultado el 5 de agosto de 2026.
8. Free Software Foundation, *seq: Print Numeric Sequences*. GNU Coreutils Manual. Disponible en: https://www.gnu.org/software/coreutils/manual/html_node/seq-invocation.html. Consultado el 5 de agosto de 2026.
9. Cervera Zafra, C., *Problemas y aplicaciones de sucesiones recurrentes*. Trabajo Fin de Máster, Universidad de Granada, Granada (2017).
10. Marecek, L., Honeycutt Mathis, A., *Intermediate Algebra 2e*. OpenStax, Houston, Texas (2020). Sección 12.2, Arithmetic Sequences.
11. Abramson, J., *College Algebra 2e*. OpenStax, Houston, Texas (2021). Sección 9.4, Series and Their Notations.

12. cppreference.com, *Fundamental types*. C++ Language. Disponible en: <https://en.cppreference.com/w/cpp/language/types>. Consultado el 7 de agosto de 2026.
13. Castro Martínez, A., *Complejidad*. Diapositivas de clase, Pontificia Universidad Javeriana (2026).
14. Sedgewick, R., Wayne, K., *Priority Queues and Heapsort*. Algorithms, 4th Edition, Princeton University. Disponible en: <https://algs4.cs.princeton.edu/24pq/>. Consultado el 8 de agosto de 2026.
15. Sedgewick, R., Wayne, K., *Quicksort*. Algorithms, 4th Edition, Princeton University. Disponible en: <https://algs4.cs.princeton.edu/23quicksort/>. Consultado el 8 de agosto de 2026.
16. Sedgewick, R., Wayne, K., *Bubble Sort*. Algorithms, 4th Edition, Princeton University. Disponible en: <https://algs4.cs.princeton.edu/21elementary/Bubble.java.html>. Consultado el 8 de agosto de 2026.